

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
DEPARTAMENTO DE INFORMÁTICA APLICADA
CURSO DE ESPECIALIZAÇÃO EM ENGENHARIA DE SOFTWARE PARA
APLICAÇÕES DE CIÊNCIA DE DADOS

ADRIANO GEBERT GOMES

**Uso de Modelos de Linguagem de Grande
Escala (LLMs) para Automação da Geração
de Casos de Testes Unitários em Software**

Monografia de Conclusão de Curso apresentada
como requisito parcial para a obtenção do grau
de Especialista em Engenharia de Software para
Aplicações de Ciência de Dados

Orientadora: Prof^a. Dra. Karina Kohl

Porto Alegre
2026

CIP — CATALOGAÇÃO NA PUBLICAÇÃO

Gomes, Adriano Gebert

Uso de Modelos de Linguagem de Grande Escala (LLMs) para Automação da Geração de Casos de Testes Unitários em Software / Adriano Gebert Gomes. – Porto Alegre: Curso de Especialização em Engenharia de Software para Aplicações de Ciência de Dados da UFRGS, 2026.

43 f.: il.

Monografia (especialização) – Universidade Federal do Rio Grande do Sul. Curso de Especialização em Engenharia de Software para Aplicações de Ciência de Dados, Porto Alegre, BR-RS, 2026. Orientadora: Karina Kohl.

1. Testes unitários. 2. Modelos de Linguagem de Grande Escala. 3. Inteligência Artificial Generativa. I. Kohl, Karina. II. Título.

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

Reitora: Prof^a. Marcia Barbosa

Vice-Reitor: Prof. Pedro de Almeida Costa

Pró-Reitora de Pós-Graduação: Prof^a. Claudia Wasserman

Diretor do Instituto de Informática: Prof. Luciano Paschoal Gaspar

Coordenadores da Especialização: Prof^a. Dra. Karin Becker e Prof. Dr. Marcelo Pimenta

Bibliotecário-chefe do Instituto de Informática: Alexsander Borges Ribeiro

*“Se vi mais longe,
foi por estar sobre ombros de gigantes.”*

— ISAAC NEWTON

AGRADECIMENTOS

Primeiramente, agradeço a Deus pelo dom da vida. Agradeço à minha família pela compreensão e apoio ao longo desta jornada.

Agradeço à minha orientadora, Professora Karina, pela aceitação da orientação, pela disponibilidade, compreensão e pelas orientações fundamentais que guiaram a construção deste trabalho de conclusão. Também agradeço à coordenadora, Professora Karin Becker, pela liderança e condução na organização do curso de especialização.

Registro minha gratidão aos professores da Universidade Federal do Rio Grande do Sul, que, ao longo deste curso, contribuíram de forma significativa para o meu aprendizado nas áreas de Ciência de Dados, Inteligência Artificial e Engenharia de Software.

Agradeço também aos colegas que conheci ao longo do curso. Em especial ao Guilherme, Luigi e Henrique pelo companheirismo e pelo trabalho em equipe desenvolvido durante as atividades em grupo.

Por fim, agradeço ao Programa de Pós-Graduação em Engenharia de Software da Universidade Federal do Pampa, no qual atualmente curso o Mestrado Profissional em Engenharia de Software. A linha de pesquisa em que atuo proporcionou experiências com o uso de Inteligência Artificial Generativa, as quais também contribuíram para o desenvolvimento deste trabalho de especialização na UFRGS.

RESUMO

O planejamento e a realização de testes de *software* desempenham papel fundamental para garantir a qualidade de sistemas, considerando aspectos como confiabilidade, usabilidade, desempenho e tolerância a falhas. Contudo, as atividades envolvendo testes de *software* demandam esforços e custos significativos, além de desafios para a obtenção de níveis adequados de cobertura. Neste contexto, este trabalho investigou o uso de LLMs no apoio à geração automática de casos de teste unitários, com o objetivo de reduzir o esforço e ampliar o nível de cobertura. Para esse fim, foi desenvolvida uma ferramenta MVP em linguagem Python para apoio à geração automática de testes utilizando o modelo de LLM Gemini 2.0. A validação da proposta foi realizada por meio de um estudo de caso aplicado a um sistema de agendamento de clínica veterinária, desenvolvido em linguagem PHP utilizando o *framework* Yii 2.0. A avaliação considerou como métricas de comparação entre a abordagem manual e a abordagem automatizada com o apoio da LLM, o tempo de geração dos casos de teste e as taxas de cobertura obtidas, incluindo as coberturas de linhas e métodos. Os resultados indicaram que a ferramenta apresentou desempenho superior no tempo de geração automática dos casos de teste com apoio da LLM, quando comparada ao processo de desenvolvimento manual. Contudo, limitações observadas durante o experimento sugeriram que o uso de inteligência artificial não substituiu o papel do profissional de tecnologia da informação, mas se mostrou como um complemento para reduzir esforços, custos e contribuir para a melhoria da qualidade das aplicações de software Web.

Palavras-chave: Testes unitários. Modelos de Linguagem de Grande Escala. Inteligência Artificial Generativa.

Using Large Language Models (LLMs) to Automate the Generation of Unit Test Cases in Software

ABSTRACT

The planning and execution of software testing play a fundamental role in ensuring the quality of systems, considering aspects such as reliability, usability, performance, and fault tolerance. However, activities related to software testing involve significant effort and costs, in addition to challenges in achieving adequate levels of coverage. In this context, this work investigated the use of LLMs to support the automatic generation of unit test cases, with the objective of reducing effort and increasing coverage levels. For this purpose, an MVP tool was developed in the Python programming language to support the automatic generation of tests using the Gemini 2.0 LLM model. The validation of the proposed approach was carried out through a case study applied to a veterinary clinic appointment scheduling system, developed in the PHP programming language using the Yii 2.0 framework. The evaluation considered, as comparison metrics between the manual approach and the automated approach supported by the LLM, the test case generation time and the coverage rates obtained, including line and method coverage. The results indicated that the tool achieved superior performance in terms of automated test case generation time when compared to the manual development process. However, limitations observed during the experiment suggested that the use of artificial intelligence does not replace the role of the information technology professional, but rather serves as a complementary approach to reduce effort and costs and to contribute to improving the quality of web-based software applications.

Keywords: Unit tests. Large Language Models. Generative Artificial Intelligence.

LISTA DE ABREVIATURAS E SIGLAS

API	<i>Application Programming Interface</i>
CLI	<i>Command Line Interface</i>
DSR	<i>Design Science Research</i>
GenIA	<i>Generative Artificial Intelligence</i>
Fatus	Ferramenta de Automatização de Testes Unitários
IA	Inteligência Artificial
JSON	<i>JavaScript Object Notation</i>
LLM	<i>Large Language Model</i>
MVC	<i>Model View Controller</i>
MVP	<i>Minimum Viable Product</i>
SACV	Sistema de Agendamento de Clínica Veterinária
TCC	Trabalho de Conclusão de Curso

LISTA DE SÍMBOLOS

t_{manual}	Tempo de desenvolvimento manual
t_{LLM}	Tempo de geração automatizada com uso da LLM
F	Fator de Aceleração
G	Ganho Percentual

LISTA DE FIGURAS

Figura 3.1	Etapas da metodologia mista adotada.....	19
Figura 4.1	Tela Fatus - lista de comandos e opções	23
Figura 4.2	Diagrama de arquitetura da solução	25
Figura 4.3	Prompt com instruções para geração de teste	27
Figura 4.4	Tela de consultas agendadas do SACV	29
Figura 4.5	Diagrama do fluxo do sistema SACV e da ferramenta Fatus	29
Figura 4.6	Tela Fatus - execução dos testes no Codeception	30
Figura 5.1	Comparação do tempo de implementação e geração de casos de teste	33
Figura 5.2	Comparação da cobertura de linhas de código-fonte.....	34
Figura 5.3	Comparação da cobertura de métodos	34
Figura A.1	Arquivo de Configuração do Fatus	43

LISTA DE TABELAS

Tabela 4.1	Parâmetros do modelo Gemini-2.0-flash utilizados	26
Tabela 5.1	Métricas de desempenho de tempo.....	32
Tabela 5.2	Comparação LLM e Desenvolvimento Manual.....	33

SUMÁRIO

1 INTRODUÇÃO	13
1.1 Objetivo Geral	13
1.1.1 Objetivos Específicos	13
1.2 Estrutura do Trabalho	14
1.3 Uso de Inteligência Artificial Generativa	14
2 REFERENCIAL TEÓRICO	15
2.1 Testes de <i>Software</i>	15
2.2 Modelos de Linguagem de Grande Escala	15
2.3 Trabalhos Relacionados	16
2.4 Ferramentas de Geração de Testes Unitários	17
2.4.1 TestSpark	17
2.4.2 TestPilot	17
2.4.3 Considerações das ferramentas	18
3 METODOLOGIA	19
3.1 <i>Design Science Research</i>	19
3.1.1 Fase <i>Awareness</i>	19
3.1.2 Fase de Design e Desenvolvimento	20
3.1.3 Fase de <i>Evaluation</i>	20
3.1.3.1 Critérios de Avaliação	21
3.1.3.2 Métricas de desempenho	22
3.1.4 Fase de <i>Improvement</i>	22
4 DESENVOLVIMENTO	23
4.1 Visão Geral da Ferramenta Proposta	23
4.2 Levantamento de Requisitos da Ferramenta	24
4.3 Práticas de desenvolvimento	24
4.4 Arquitetura da Solução Proposta	25
4.5 Processo de Geração de Testes	25
4.5.1 Ferramenta de IA Generativa	26
4.5.2 Estrutura dos prompts enviados ao LLM	26
4.5.3 Estratégia de coleta das respostas	26
4.5.4 Pós-processamento e adaptação para PHPUnit	27
4.6 Arquivo de Configuração	27
4.7 Estudo de caso: SACV	28
4.7.1 Seleção dos Módulos para Teste	28
4.7.2 Ambiente de execução	30
4.7.3 Execução dos testes unitários	30
5 RESULTADOS	32
5.1 Análise Quantitativa	32
5.1.1 Tempo de Desenvolvimento e Geração dos Testes	32
5.1.2 Comparação da Cobertura de Código	33
5.2 Análise Qualitativa	35
5.2.1 Considerações do experimento	35
5.2.2 Componente Helper	36
5.2.3 Componente Consulta	36
5.2.4 Componente Login	38
6 DISCUSSÃO	40
7 CONCLUSÕES	41
REFERÊNCIAS	42

APÊNDICE A — ARQUIVO DE CONFIGURAÇÃO.....	43
--	-----------

1 INTRODUÇÃO

Atualmente os programas de computador fazem parte do cotidiano das pessoas e organizações e, como produtos de *software* com valor agregado ao cliente, espera-se que apresentem elevado nível de qualidade, incluindo confiabilidade, usabilidade e bom desempenho. Nesse contexto, a realização de testes desempenha papel fundamental na identificação de defeitos e no aumento da robustez do *software*.

Entretanto, a realização de testes de software envolve custos e esforços significativos, que contemplam as fases de planejamento, especificação, desenvolvimento de *software* e a execução dos testes, além dos desafios para obter níveis adequados de cobertura (Ellims; Bridges; Ince, 2006). Embora existam ferramentas de automação de testes, ainda são necessários esforços na criação dos casos de teste que dependem fortemente do trabalho humano e da experiência do testador (Ricca et al., 2025).

Com o crescente uso de *Large Language Models* (LLMs) para aumentar a produtividade em atividades de engenharia de *software*, surge a oportunidade de investigar sua aplicação no apoio à geração automática de casos de teste de *software*, com o objetivo de reduzir o esforço associado a esta atividade e, potencialmente, ampliar a cobertura de testes e a qualidade do *software* resultante.

1.1 Objetivo Geral

Investigar o uso de LLMs no apoio à geração de casos de teste de *software*, por meio de um estudo de caso aplicado a um sistema PHP desenvolvido pelo autor.

Para alcançar esse objetivo, o trabalho propôs o desenvolvimento de um *Minimum Viable Product* (MVP) para apoio à geração automática de casos de teste usando LLMs. A validação da proposta foi realizada por meio da comparação entre os casos de teste gerados manualmente e os casos de teste gerados com apoio do MVP, considerando métricas de tempo de desenvolvimento, cobertura e qualidade dos testes obtidos.

1.1.1 Objetivos Específicos

- Aplicar LLMs no apoio à geração de casos de teste de *software*;
- Desenvolver um MVP para análise do código-fonte e apoio à geração de casos de

teste com o uso de LLMs;

- Avaliar a cobertura e qualidade dos casos de teste gerados com apoio do MVP;
- Comparar o esforço, o tempo de desenvolvimento e os resultados em relação aos casos de teste criados manualmente.

1.2 Estrutura do Trabalho

Este trabalho de conclusão da Especialização em Engenharia de Software para Aplicações de Ciência de Dados foi estruturado em sete capítulos. No capítulo 2 são apresentados os conceitos relevantes sobre testes de *software* e LLMs, bem como os trabalhos relacionados e ferramentas de apoio à geração de testes de *software* utilizando LLMs. No capítulo 3 é apresentada a metodologia adotada neste trabalho, detalhando as fases previstas e os critérios de aceitação adotados. No capítulo 4 são apresentados o processo de desenvolvimento da ferramenta de apoio à geração de testes de *software* e o repositório utilizado como caso de estudo para este trabalho.

No capítulo 5 são apresentados os resultados obtidos durante a execução da ferramenta desenvolvida. No capítulo 6 são apresentadas discussões das principais impressões e análises decorrentes do desenvolvimento da ferramenta e da execução dos testes. No capítulo 7 são apresentadas as conclusões deste trabalho.

1.3 Uso de Inteligência Artificial Generativa

No desenvolvimento deste trabalho foram utilizadas ferramentas de inteligência artificial generativa, como ChatGPT e NotebookLM, para organização da estrutura, revisão textual e análise crítica do conteúdo escrito, cabendo ao autor o uso ou não das sugestões propostas.

A autoria intelectual deste trabalho, incluindo a definição do problema, o método adotado, a interpretação dos resultados e a redação final, foi de responsabilidade do autor.

2 REFERENCIAL TEÓRICO

Neste capítulo são apresentados alguns conceitos relevantes e tecnologias utilizadas para o desenvolvimento deste trabalho.

2.1 Testes de *Software*

O desenvolvimento de *software* como produto necessita de controles de qualidade e confiabilidade para seu uso pelas pessoas, de modo muito semelhante a outros produtos existentes. No entanto, um *software* difere em termos de complexidade e que, mesmo em uso, está em constante mudança e, nesse cenário, a execução de testes de *software* é fundamental para evitar falhas e erros nas aplicações (Valente, 2020). Considerando a importância dos testes e questões envolvendo custos e produtividade, hoje existem ferramentas que possibilitam a automatização dos testes de *software* através da criação de *scripts* de casos de teste a serem executados (Valente, 2020).

Considerando os testes automatizados, eles podem ser classificados em **testes unitários** que são referentes à implementação a nível de código-fonte em métodos e classes, **testes de integração** que são referentes à integração de APIs e módulos do mesmo e/ou diferentes sistemas e *testes de sistema* que são referentes à interação do usuário com a interface da aplicação (Valente, 2020).

2.2 Modelos de Linguagem de Grande Escala

Os Modelos de Linguagem de Grande Escala, em inglês *Large Language Models*, são modelos com milhões ou bilhões de parâmetros e grande volume de dados que foram treinados utilizando Aprendizado Profundo, com o uso de redes neurais e *transformers* que possibilitam que o usuário converse naturalmente com o modelo por meio da linguagem natural (Kausik, 2026). As LLMs são uma subcategoria da Inteligência Artificial Generativa, em que, para uma determinada entrada de texto do usuário, denominada aqui de *prompt*, é predita pela IA a próxima sequência de texto a ser apresentada ao usuário, como se fosse uma conversa com outra pessoa sobre determinado assunto (Ramos, 2023).

2.3 Trabalhos Relacionados

Nesta seção são apresentados trabalhos existentes na literatura que estão relacionados à geração de testes automatizados com o uso de Inteligência Artificial Generativa.

A utilização dos LLMs para a automação de atividades próprias da Engenharia de Software vem sendo amplamente estudada e aplicada com vistas ao aumento da produtividade e da qualidade em diversas tarefas, como geração de código-fonte, desenvolvimento de interfaces e apoio à realização de testes de *software* (Schmidt, 2025). Em especial, a geração de casos de teste por meio desses LLMs tem sido explorada na literatura recente e na indústria, e apresenta potencial para melhoria da qualidade do código-fonte e para o aumento da cobertura dos testes realizados quando comparados ao desenvolvimento manual (Eldh, 2025).

Para mitigar o esforço e os custos associados à atividade de implementação de testes unitários, diversos trabalhos vêm investigando o uso de LLMs para automatização da geração de testes unitários (Schmidt, 2025; Lops et al., 2025). Nessa linha, Lops et al. (2025) propõe o AGONETEST, um sistema automatizado para geração de testes unitários com o uso de LLMs, e conduz um experimento comparando o processo manual de implementação de casos de teste com os casos de teste gerados pela ferramenta. A avaliação utilizou métricas de cobertura de código, como linhas e métodos, e os resultados apresentaram taxas de 75% de execução dos testes gerados, sendo que cerca de 35% foram aprovados (Lops et al., 2025). Os resultados obtidos sugerem que as LLMs possuem capacidade para geração de casos de teste, mas ainda carecem de maior assertividade, devendo as LLMs ser utilizadas como ferramenta de apoio, e não substituir o trabalho realizado por um profissional humano (Lops et al., 2025).

Apesar das vantagens observadas no uso de LLMs para a geração de testes unitários, diversos desafios e limitações são discutidos. Entre eles destacam-se a assertividade dos casos de teste gerados, a necessidade de maior contexto da aplicação para geração de casos de teste adequados, o consumo energético, além da existência de vieses e de alucinações dos modelos (Ricca et al., 2025; Eldh, 2025; Schmidt, 2025). Além disso, persistem questões éticas e de privacidade relacionadas ao uso desses LLMs, uma vez que códigos-fonte e dados associados podem ser processados por serviços externos, o que pode gerar preocupações quanto ao uso desse código-fonte para treinamento dos modelos ou possíveis vazamentos de dados (Ricca et al., 2025).

Sendo assim, o uso de ferramentas baseadas em LLMs não deve substituir uma

pessoa humana, mas sim complementar as atividades inerentes à realização de casos de teste, bem como à validação final por uma pessoa (Schmidt, 2025). Nesse contexto, este trabalho propõe a implementação de uma ferramenta que possibilite a geração de casos de teste unitários utilizando um modelo de LLM, avaliando a assertividade e a qualidade dos testes por meio de métricas quantitativas e qualitativas.

2.4 Ferramentas de Geração de Testes Unitários

Nesta seção serão apresentadas duas ferramentas que utilizam Inteligência Artificial para a geração de testes de *software*.

2.4.1 TestSpark

A TestSpark é um *plugin* que foi desenvolvido para a IDE JetBrains, que possibilita a geração de testes unitários para a linguagem Java (JetBrains, 2026). O acesso a esta ferramenta é gratuito. O plugin possui três estratégias para a geração das suítes de testes: (i) LLM-Based Test Generate: uso da LLM para geração dos casos de teste, (ii) Search-Local: uso de busca local para geração dos testes, (iii) Execução simbólica: uso dos *bytecodes* da aplicação Java para geração dos casos de teste, nesse caso, eles não são muito legíveis (JetBrains, 2026). Para a estratégia usando LLM, o TestSpark envia um *prompt* para a LLM e a suíte de teste gerada é apresentada ao usuário que decide se deve ou não aceitar. Conforme a documentação da ferramenta, a mesma está em fase experimental e é recomendado que seja utilizada para aumentar a suíte de testes (JetBrains, 2026).

2.4.2 TestPilot

O TestPilot 2 é uma ferramenta desenvolvida para a geração de testes unitários para os pacotes **npm**, que são implementados usando as linguagens JavaScript e TypeScript. A ferramenta possui um GitHub onde é disponibilizada gratuitamente e de código aberto, sob os termos da licença MIT. O TestPilot 2 utiliza a API de LLMs para geração dos casos de teste, permitindo que o usuário escolha o modelo de LLM a seu critério. O *pipeline* adotado pela ferramenta é o envio de *prompt* contendo alguns dados do código-

fonte como corpo de métodos, assinaturas, comentários para a LLM. O retorno da LLM é traduzido para uma suíte de teste executável, e a aplicação realiza essa execução. Em caso de falhas, é enviada uma nova requisição à LLM contendo a suíte de teste gerada e o erro apresentado, possibilitando o refinamento da suíte de teste (TestPilot, 2026). A ferramenta é disponibilizada de forma experimental, conforme documentação do GitHub (TestPilot, 2026).

2.4.3 Considerações das ferramentas

As ferramentas apresentadas aqui como trabalhos relacionados, a TestSpark e o TestPilot, permitem a geração de casos de teste unitários utilizando as LLMs. A TestSpark possui algumas estratégias adicionais para aumentar o seu desempenho. Ambas as ferramentas são gratuitas, de código aberto e possuem ressalvas de caráter experimental. Algo em comum também é que as ferramentas não são generalistas, elas são específicas para uma determinada linguagem de programação.

Analisando a documentação das ferramentas, fica evidente que essas ferramentas são suscetíveis a erros e sugerem que o propósito não é substituir, mas sim aperfeiçoar as suítes de testes existentes nas aplicações.

3 METODOLOGIA

Neste capítulo é apresentada a metodologia adotada no desenvolvimento deste trabalho.

3.1 *Design Science Research*

O paradigma *Design Science Research* (DSR) é uma abordagem adotada para a construção de artefatos com potencial de inovação (e.g. aplicações, protótipos, ferramentas, MVP) que permitem solucionar problemas reais de maneira prática (Marcon; Favarim, 2021).

No contexto deste trabalho, essa abordagem possibilitou a construção e avaliação de um artefato aplicado à geração de casos de teste de *software*. Os princípios adotados conduzem a pesquisa para a resolução de problemas em contexto real (Ralph et al., 2020).

Para a condução deste trabalho, foi realizada uma adaptação do paradigma DSR, estruturada de forma sequencial nas fases ilustradas na Figura 3.1 que são descritas nas próximas seções deste capítulo.

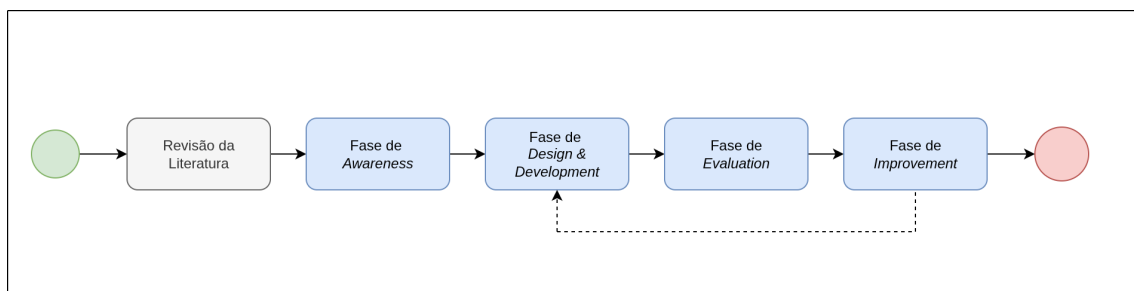


Figura 3.1 – Etapas da metodologia mista adotada

3.1.1 Fase *Awareness*

Nesta fase foi realizado o reconhecimento do problema e dos objetivos a serem alcançados com a implementação do artefato para automatização dos testes de *software*. Para este trabalho, foi identificado como problema a dificuldade do desenvolvimento manual dos casos de teste, devido ao custo envolvido e à baixa cobertura e qualidade.

3.1.2 Fase de Design e Desenvolvimento

Nesta fase foi elaborado o artefato de pesquisa, materializado em um *Minimum Viable Product* (MVP) desenvolvido em linguagem Python e integrado a um Modelo de Linguagem de Grande Porte (LLM). O MVP teve como objetivo apoiar a geração automatizada de casos de teste a partir de código-fonte existente e foi composto por três módulos principais, descritos a seguir:

- a) **Módulo de Mineração:** responsável pela extração de trechos de código e de testes já existentes no repositório do projeto. Essa mineração foi restrita ao contexto de testes de unidade, estruturando as informações extraídas em formato JSON para uso posterior pelos demais módulos;
- b) **Módulo de Integração com LLM:** responsável pela comunicação com o modelo de linguagem por meio de chamadas à sua API. Esse módulo utilizou como entrada a estrutura JSON gerada pela mineração, definindo e enviando os *prompts* necessários para que o LLM produzisse casos de teste correspondentes às funções ou classes analisadas;
- c) **Módulo de Validação Automatizada:** encarregado de executar os casos de teste gerados pelo LLM e de registrar métricas de qualidade, como cobertura de código e sucesso na execução. Os resultados obtidos serviram como base para a avaliação do artefato.

Concluída a fase de *design* e desenvolvimento, o artefato resultante apresentou uma arquitetura funcional capaz de integrar os três módulos descritos, possibilitando a extração de informações de código-fonte, a comunicação automatizada com o modelo de LLM e a execução dos casos de teste unitários gerados. Essa implementação da ferramenta permitiu verificar a viabilidade técnica da abordagem proposta e serviu de base para a etapa seguinte da pesquisa, dedicada à avaliação do artefato quanto à sua eficácia, desempenho e contribuição prática para a geração automatizada de testes unitários.

3.1.3 Fase de *Evaluation*

Nesta fase foi realizada a validação do artefato utilizando um projeto piloto previamente selecionado em repositório. Neste momento, foi realizado o desenvolvimento de

casos de teste manuais e a geração de casos de teste automatizados pelo artefato.

As métricas adotadas para essa validação foram quantitativas e qualitativas. Sendo as métricas quantitativas: i) tempo para geração de testes, ii) percentual de cobertura de código antes e depois da validação, iii) *bugs* detectados durante a execução dos casos de teste; e as qualitativas: i) *Feedback* do desenvolvedor sobre as impressões com o uso da ferramenta e dos resultados obtidos.

A fase de avaliação teve como objetivo verificar a eficácia e a aplicabilidade do artefato desenvolvido, considerando sua capacidade de apoiar a geração automatizada de casos de teste unitários e de contribuir para a redução do esforço manual no processo de validação de *software*. Seguindo os princípios da *Design Science Research* (DSR), a avaliação buscou observar tanto a qualidade técnica do artefato implementado quanto sua utilidade prática no contexto da geração e execução dos casos de teste unitários.

Para essa etapa, foram definidos critérios quantitativos e qualitativos de análise. Entre as métricas quantitativas, consideraram-se a cobertura de código obtida pelos testes gerados, o número de falhas detectadas e o tempo de execução da geração dos casos de teste unitários em comparação ao processo manual. Já a avaliação qualitativa concentrou-se na clareza, relevância e assertividade dos casos de teste produzidos pelo modelo de LLM, observadas a partir de análise exploratória e de validação pelo desenvolvedor envolvido no projeto.

Os experimentos foram conduzidos utilizando o código-fonte do sistema de referência desenvolvido durante o trabalho de conclusão da graduação do autor, permitindo avaliar o desempenho do artefato em uma aplicação já desenvolvida e validada com usuários.

3.1.3.1 Critérios de Avaliação

Os critérios que foram adotados neste trabalho compreendem o desenvolvimento manual dos casos de teste, a geração automatizada dos casos de teste pela LLM e a cobertura obtida pelos casos de teste.

Na avaliação do desenvolvimento manual dos testes foram contemplados o tempo necessário para elaboração dos casos de teste e o número de testes desenvolvidos.

Na avaliação da geração automatizada dos testes foram contemplados o tempo necessário de geração, o número de testes desenvolvidos e a executabilidade dos testes na ferramenta Codeception. Adicionalmente, foi inspecionada, de forma qualitativa, a assertividade dos testes gerados.

Em ambas as abordagens foram avaliadas a cobertura de linhas obtida na execução dos casos de teste pela ferramenta Codeception.

3.1.3.2 Métricas de desempenho

Para avaliação de desempenho foram adotadas as métricas de Ganho de Percentual e Fator de Desempenho. A métrica de Ganho de Percentual, conforme a Equação 3.1, permitiu a análise de ganho da LLM em relação à abordagem manual.

$$\text{Ganho Percentual } (G) = \left(\frac{t_{\text{manual}} - t_{\text{LLM}}}{t_{\text{manual}}} \right) \times 100 \quad (3.1)$$

A métrica de Fator de Desempenho, conforme a Equação 3.2, permitiu a análise do número de vezes que a LLM superou a abordagem Manual.

$$\text{Fator de Aceleração } (A) = \frac{t_{\text{manual}}}{t_{\text{LLM}}} \quad (3.2)$$

Sendo a variável t_{manual} o tempo de desenvolvimento manual dos casos de teste e a variável t_{LLM} o tempo de geração dos casos de teste com uso da LLM pela ferramenta. As métricas acima foram aplicadas individualmente para cada componente, no caso, a classe contendo os métodos que foram testados.

3.1.4 Fase de *Improvement*

Nesta fase foram identificadas e implementadas melhorias no artefato desenvolvido, com o objetivo de aprimorar seu desempenho e corrigir limitações observadas durante a avaliação dos testes unitários executados. As principais melhorias envolveram o refinamento das regras de *prompting*, a partir da análise das falhas recorrentes geradas pelo modelo de LLM, bem como ajustes nos mecanismos de integração com ferramentas externas utilizadas na execução e validação dos testes automatizados. Além disso, foram realizados ajustes na base de dados utilizada pela aplicação de referência. Essas modificações contribuíram para aumentar a consistência das respostas do LLM e a estabilidade do processo automatizado de geração e execução de casos de teste unitários.

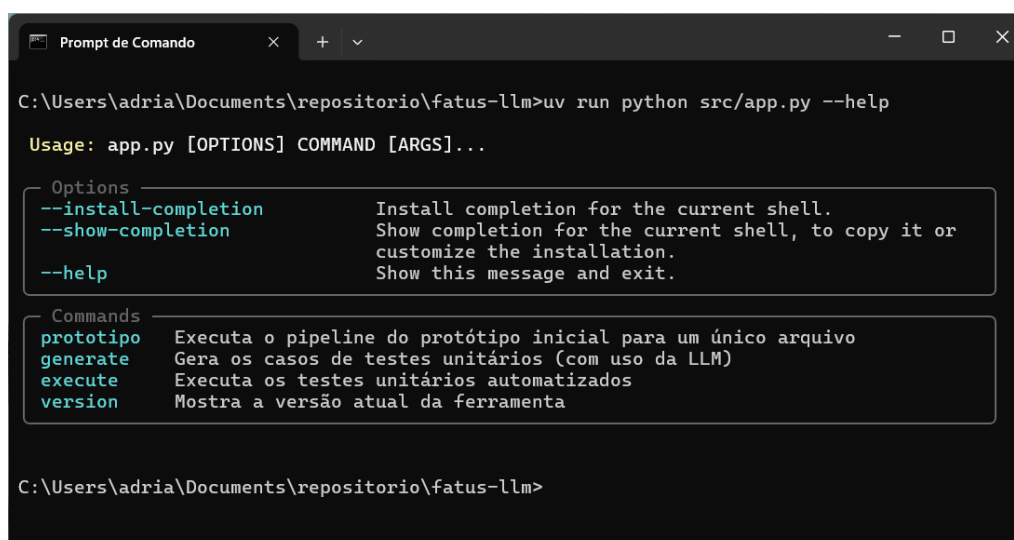
4 DESENVOLVIMENTO

Neste capítulo é descrito o processo de desenvolvimento da ferramenta para apoio à geração de testes unitários com o uso de uma API de LLMs, conforme proposto na metodologia deste trabalho na Subseção 3.1.2. Ao final, é apresentado o Sistema de Agendamento de Clínica Veterinária, utilizado como caso de estudo para validação da geração e execução dos testes unitários.

4.1 Visão Geral da Ferramenta Proposta

A Ferramenta de Automatização de Testes Unitários (Fatus) é uma aplicação em linguagem Python que constitui o artefato materializado do MVP proposto na Subseção 3.1.2. A Fatus tem como principal objetivo servir de apoio à geração de testes unitários, tendo como entrada o código-fonte PHP para análise e possibilitando a integração com uma ferramenta de automatização de testes, como Codeception, para execução dos casos de teste gerados com uso da LLM. A Fatus foi desenvolvida utilizando a biblioteca Typer, que facilita o desenvolvimento de aplicações Python que utilizam a interface CLI. Na Figura 4.1 é possível conferir a captura de tela da ferramenta com as opções e comandos disponíveis, no estilo usual da biblioteca Typer.

Figura 4.1 – Tela Fatus - lista de comandos e opções



```
C:\Users\adria\Documents\repositorio\fatus-llm>uv run python src/app.py --help

Usage: app.py [OPTIONS] COMMAND [ARGS]...

Options
  --install-completion  Install completion for the current shell.
  --show-completion     Show completion for the current shell, to copy it or
                        customize the installation.
  --help                Show this message and exit.

Commands
  prototipo  Executa o pipeline do protótipo inicial para um único arquivo
  generate   Gera os casos de testes unitários (com uso da LLM)
  execute    Executa os testes unitários automatizados
  version    Mostra a versão atual da ferramenta

C:\Users\adria\Documents\repositorio\fatus-llm>
```

Fonte: próprio autor

4.2 Levantamento de Requisitos da Ferramenta

Para o levantamento dos requisitos, foi adotada como técnica de engenharia de requisitos as histórias do usuário. Assim, foram definidas as seguintes histórias para o desenvolvimento das funcionalidades básicas da ferramenta Fatus:

- Como desenvolvedor quero fornecer um repositório de código-fonte para a ferramenta gerar casos de teste unitários.
- Como desenvolvedor quero visualizar os casos de teste unitários gerados.
- Como desenvolvedor quero executar os casos de teste gerados pela ferramenta.
- Como desenvolvedor quero visualizar o resultado obtido na execução dos casos de teste.
- Como desenvolvedor quero visualizar a cobertura obtida dos casos de teste.

4.3 Práticas de desenvolvimento

Para o desenvolvimento da ferramenta Fatus, foi criado um repositório Git para versionamento das alterações do código-fonte. Essa prática permite o controle das funcionalidades implementadas ao longo do processo de desenvolvimento, possibilitando a rastreabilidade das modificações. Para este trabalho foi utilizado um repositório privado do GitHub.

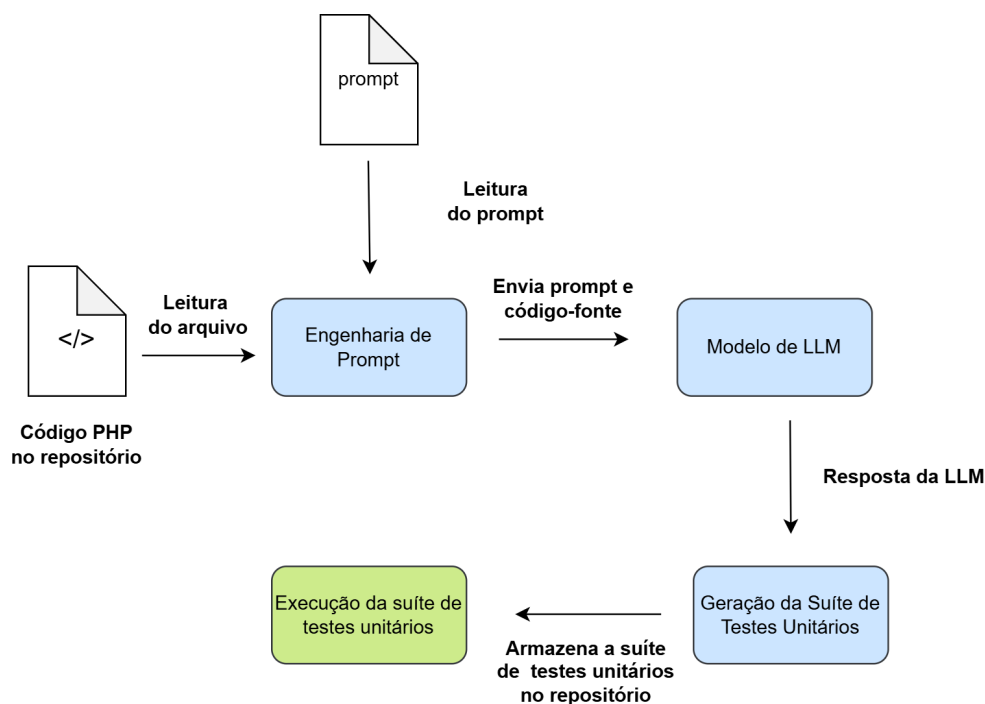
Além disso, foi utilizado o gerenciador de dependências *uv*, que permite o controle das versões específicas das bibliotecas utilizadas, assim como a criação de um ambiente virtual isolado, evitando interferências com outras aplicações Python. O uso desse gerenciador contribui para evitar conflitos de dependências e facilita a execução da ferramenta em diferentes ambientes computacionais.

Por fim, a implementação do código-fonte foi realizada com a IDE Visual Studio Code, devido aos recursos disponíveis, como extensões e formatação de código, que auxiliam na produtividade.

4.4 Arquitetura da Solução Proposta

Na Figura 4.2, é apresentada a arquitetura da solução da ferramenta Fatus, tendo como entrada a leitura do código PHP do repositório que, posteriormente, é concatenado com o texto de *prompt* definido. A próxima fase consiste no envio para análise do modelo de LLM que retorna o código da suíte de testes unitários gerados. O retorno do código passa por um pós-processamento para ajustes complementares antes da gravação em arquivo PHP que será executado pela ferramenta de testes unitários. Nesse caso, o arquivo é executado pelo Codeception.

Figura 4.2 – Diagrama de arquitetura da solução



Fonte: próprio autor

A arquitetura proposta permite um fluxo contínuo que compreende a leitura do código-fonte, a comunicação com a LLM para análise e geração dos casos de teste e a execução automatizada dos testes de forma integrada à ferramenta.

4.5 Processo de Geração de Testes

Nesta seção é detalhado o processo de geração de testes implementado pela ferramenta Fatus. São apresentadas a visão geral do modelo de LLM utilizado, a construção

do *prompt* e a estratégia utilizada para a coleta das respostas obtidas pela LLM. Por fim, foram necessários ajustes de pós-processamento e adaptações, que são apresentadas na sequência.

4.5.1 Ferramenta de IA Generativa

Para este trabalho, foi utilizado o modelo Gemini-2.0-flash disponibilizado pela empresa Google. A escolha deste modelo se deu pela facilidade no envio de *prompts* por meio de requisições utilizando API, pela possibilidade de geração de código-fonte e pelo uso gratuito.

As requisições ao modelo foram realizadas por meio da biblioteca *genai*, utilizando chamadas via API estruturadas no formato JSON. Como entrada, a aplicação envia um *prompt* contendo as instruções e o código-fonte da classe PHP a ser analisada pelo modelo. Como saída, o modelo retorna o código dos casos de teste. Neste trabalho, foram mantidos os valores padrão do modelo, conforme apresentados na Tabela 4.1.

Parâmetro	Intervalo	Padrão
temperatura	0,0 – 2,0	1,0
topP	0,0 – 1,0	0,95
topK	64	64
candidateCount	1 – 8	1

Tabela 4.1 – Parâmetros do modelo Gemini-2.0-flash utilizados

4.5.2 Estrutura dos prompts enviados ao LLM

Foi adotada uma estrutura de *prompt* contendo as instruções para a geração de caso de teste, especificando que é para o Codeception juntamente com o código-fonte do arquivo PHP. Na aplicação, foram então concatenados esse *prompt* e o conteúdo lido do arquivo PHP, conforme a Figura 4.3

4.5.3 Estratégia de coleta das respostas

Após o envio do *prompt* contendo o código-fonte, foi enviada uma requisição via API para a LLM, que retornou apenas o código do *script* de testes unitários.

Prompt com instruções para geração de teste

Você é um desenvolvedor web de aplicações feitas com o framework Yii 2.0 e testes no Codeception versão 5.3. A sua tarefa é gerar casos de teste unitários desta aplicação.

Você vai receber como entrada o código de uma classe PHP e deve gerar como saída apenas o código do TestCase. O TestCase gerado deverá considerar todas possíveis partições de entrada e obter uma cobertura elevada (pode ter mais de 50 a 100 asserts, eu espero).

Recomendações: I) No uso de banco de dados, lembre de remover os registros criados para cada teste. II) Evite o uso de fixtures, caso precise, usar apenas os atributos necessários que já existem no código.

Código de entrada ==>

Figura 4.3 – Prompt com instruções para geração de teste

4.5.4 Pós-processamento e adaptação para PHPUnit

A implementação da ferramenta permite realizar a gravação direta da resposta obtida pela LLM nos arquivos de testes que serão executados pelo Codeception. Ocorre que, durante as execuções preliminares, foram necessários alguns ajustes manuais para que os *scripts* pudessem ser executados e houve a repetição de alguns padrões.

Considerando a dificuldade de ajustar o *prompt* e como eram ajustes mais triviais, foi implementada na ferramenta uma etapa de pós-processamento do retorno da LLM para permitir a execução dos *scripts* gerados sem intervenção humana.

A fase de pós-processamento consiste na execução de dois ajustes, que são:

- a) **Remoção de marcações de código:** em todas as iterações com a LLM são geradas as marcações de início “*php*” e de fim “” para indicar que o retorno é um código-fonte PHP. Nessa etapa são removidas essas marcações.
- b) **Verificação de tag PHP:** em algumas iterações foram gerados códigos-fonte sem a tag `<?php`, inviabilizando que o *script* fosse executado. Nessa etapa, é verificado se existe uma tag PHP no arquivo; caso contrário, é feita a inclusão.

Após a realização dos ajustes de pós-processamento do retorno da LLM, é realizada a gravação do arquivo contendo o *script* de teste para ser executado no Codeception.

4.6 Arquivo de Configuração

A ferramenta Fatus faz a leitura de um arquivo de configuração *config.json* estruturado em formato JSON no seu diretório raiz. Este arquivo permite definir a localização

dos caminhos referentes ao diretório da aplicação, da ferramenta Codeception, dos componentes PHP analisados e do diretório de gravação dos testes. Um exemplo deste arquivo pode ser conferido no Apêndice A.

4.7 Estudo de caso: SACV

O Sistema de Agendamento de Clínica Veterinária (SACV) é uma aplicação web que foi um artefato produzido durante o trabalho de graduação em Ciência da Computação pelo próprio autor. Esta aplicação tem como propósito ser uma solução completa para o agendamento de consultas veterinárias, contemplando as funcionalidades de cadastro de animais, cadastro de tutores, cadastro de médicos veterinários; o agendamento de consultas; a evolução do peso do animal; o envio de e-mail do agendamento; a emissão de relatórios e os prognósticos, entre outras (Gomes, 2023).

A aplicação foi desenvolvida em linguagem PHP utilizando o *framework* Yii 2.0, que possui em suas dependências o *framework* Codeception, que é uma solução completa para testes unitários, de integração e de aceitação. Durante o desenvolvimento do SACV, no âmbito do trabalho de conclusão da graduação, não foram desenvolvidos casos de teste unitários.

A aplicação está implementada em uma arquitetura MVC, onde temos os modelos contendo os Active Records que permitem o mapeamento objeto-relacional e a lógica de negócio, Visão com os arquivos HTML e as Controllers que permitem o tratamento das requisições via rota. Além desse padrão, temos alguns elementos como componentes e *widgets* que permitem o reuso (Gomes, 2023).

Na Figura 4.4 temos uma captura de tela do sistema.

4.7.1 Seleção dos Módulos para Teste

Para o escopo deste trabalho, a aplicação SACV foi adotada como estudo de caso, servindo como repositório de entrada para avaliação da ferramenta proposta.

Para geração de casos de teste unitários manuais e automatizados, foram escolhidas as classes principais de modelo, que são **Pessoa**, **Animal**, **Relatorio**, **Login** e o componente **Helper**. Para os casos de teste que dependem de outros componentes foram utilizados *mocks* para simular o comportamento, sem violar o escopo do teste unitário,

Figura 4.4 – Tela de consultas agendadas do SACV

SACV Início Cadastro ▾ Relatórios ▾ Agendamento ▾ Sobre ▾ Logout (Veterinário 1)

Cancelar.
Para edição do prontuário e conclusão do atendimento, favor clicar em **Finalizar**.
Caso precise realizar um novo agendamento de consulta, favor clicar em **Novo agendamento**.

Minhas consultas

Novo agendamento

Exibindo **1-3** de **3** itens.

Tutor	Animal	Espécie	Data	Horário		
Tutor 1	Pet 2	Gato	29/12/2025	11:30	Cancelar	Finalizar
Tutor 2	Pet 3	Cachorro	29/12/2025	15:00	Cancelar	Finalizar
Tutor 3	Pet 5	Cachorro	30/12/2025	11:00	Cancelar	Finalizar

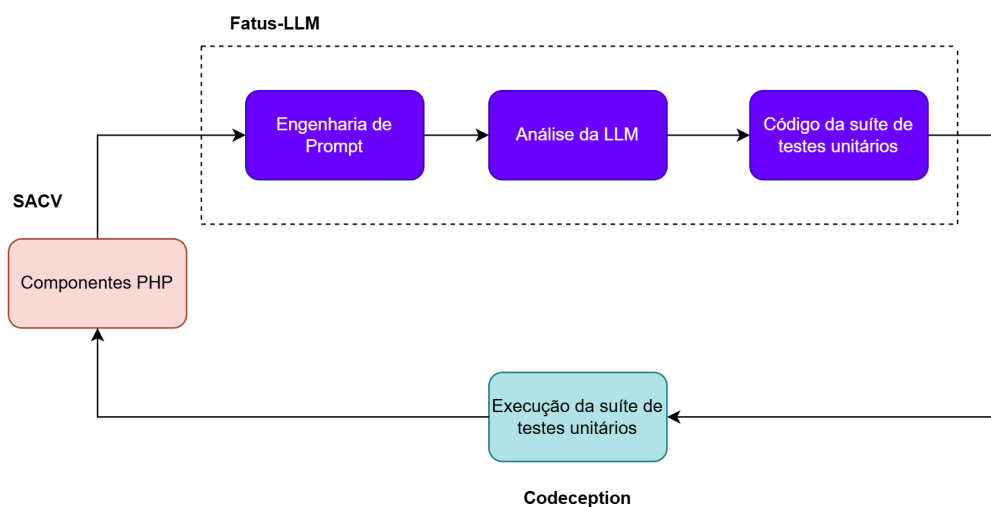
© SACV - Adriano Gebert Gomes 2025 Powered by [Yii Framework](#)

Fonte: próprio autor

que poderia acabar sendo um teste de integração.

Na Figura 4.5 temos a demonstração da interação para os componentes selecionados do SACV com a ferramenta Fatus.

Figura 4.5 – Diagrama do fluxo do sistema SACV e da ferramenta Fatus



Fonte: próprio autor

4.7.2 Ambiente de execução

Para a execução do SACV, incluindo os testes unitários no Codeception, foi necessária a instalação do pacote XAMPP que contém o servidor Apache, PHP 8.0 e Banco de Dados Relacional Maria DB.

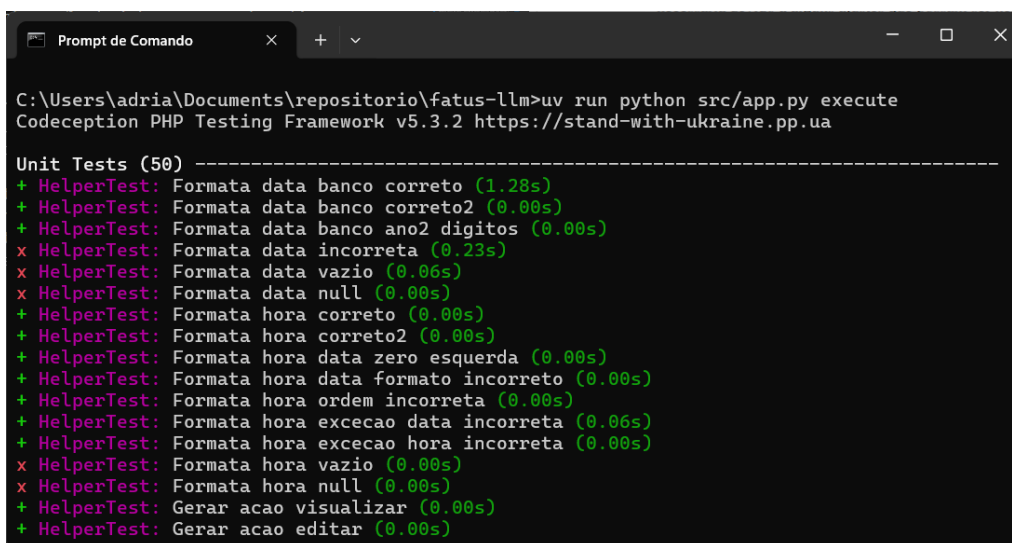
Para execução da ferramenta Fatus, foram necessários o interpretador Python 3.12 e o gerenciador de dependências uv.

4.7.3 Execução dos testes unitários

A Fatus utiliza o *framework* de testes Codeception para a execução dos testes unitários gerados. O Codeception como *framework* de testes, utiliza para a execução dos testes unitários o PHPUnit. As aplicações Web desenvolvidas utilizando o *framework* Yii 2.0, possuem em suas dependências o Codeception por padrão.

A aplicação SACV utilizada como estudo de caso foi desenvolvida em Yii 2.0, tendo o Codeception devidamente instalado na pasta *vendor*. Sendo assim, para a integração e execução dos testes por meio do Fatus, é necessário ajustar os caminhos da aplicação no arquivo de configuração e executar o comando `uv run python src/app.py execute`. Após isso, o Codeception é inicializado e a execução dos testes é apresentada conforme a captura de tela apresentada na Figura 4.6.

Figura 4.6 – Tela Fatus - execução dos testes no Codeception



```
Prompt de Comando
C:\Users\adria\Documents\repositorio\fatus-llm>uv run python src/app.py execute
Codeception PHP Testing Framework v5.3.2 https://stand-with-ukraine.pp.ua

Unit Tests (50) -----
+ HelperTest: Formata data banco correto (1.28s)
+ HelperTest: Formata data banco correto2 (0.00s)
+ HelperTest: Formata data banco ano2 digitos (0.00s)
x HelperTest: Formata data incorreta (0.23s)
x HelperTest: Formata data vazio (0.06s)
x HelperTest: Formata data null (0.00s)
+ HelperTest: Formata hora correto (0.00s)
+ HelperTest: Formata hora correto2 (0.00s)
+ HelperTest: Formata hora data zero esquerda (0.00s)
+ HelperTest: Formata hora data formato incorreto (0.00s)
+ HelperTest: Formata hora ordem incorreta (0.00s)
+ HelperTest: Formata hora execucao data incorreta (0.06s)
+ HelperTest: Formata hora execucao hora incorreta (0.00s)
x HelperTest: Formata hora vazio (0.00s)
x HelperTest: Formata hora null (0.00s)
+ HelperTest: Gerar acao visualizar (0.00s)
+ HelperTest: Gerar acao editar (0.00s)
```

Para a análise de cobertura de testes foi necessário ajustar as configurações do PHP adicionando o *plugin* XDebug e adicionar a opção *coverage* no Fatus com o comando *uv run python src/app.py execute coverage*.

5 RESULTADOS

Neste capítulo são apresentados os resultados obtidos, comparando os casos de teste desenvolvidos manualmente e aqueles gerados com o apoio de LLMs por meio da ferramenta Fatus.

5.1 Análise Quantitativa

Nesta seção são apresentados os resultados quantitativos obtidos, considerando as métricas do tempo de desenvolvimento, tempo de geração e taxa de cobertura de testes obtidas com o uso da ferramenta Fatus.

5.1.1 Tempo de Desenvolvimento e Geração dos Testes

A Tabela 5.1 apresenta a consolidação das métricas de desempenho relacionadas ao tempo de desenvolvimento manual e à geração automatizada de testes com o uso de LLMs. Os tempos de ambas as abordagens foram padronizados em minutos. O Ganho Percentual foi calculado conforme a Equação 3.1, enquanto o Fator de Aceleração foi calculado conforme a Equação 3.2.

Tabela 5.1 – Métricas de desempenho de tempo

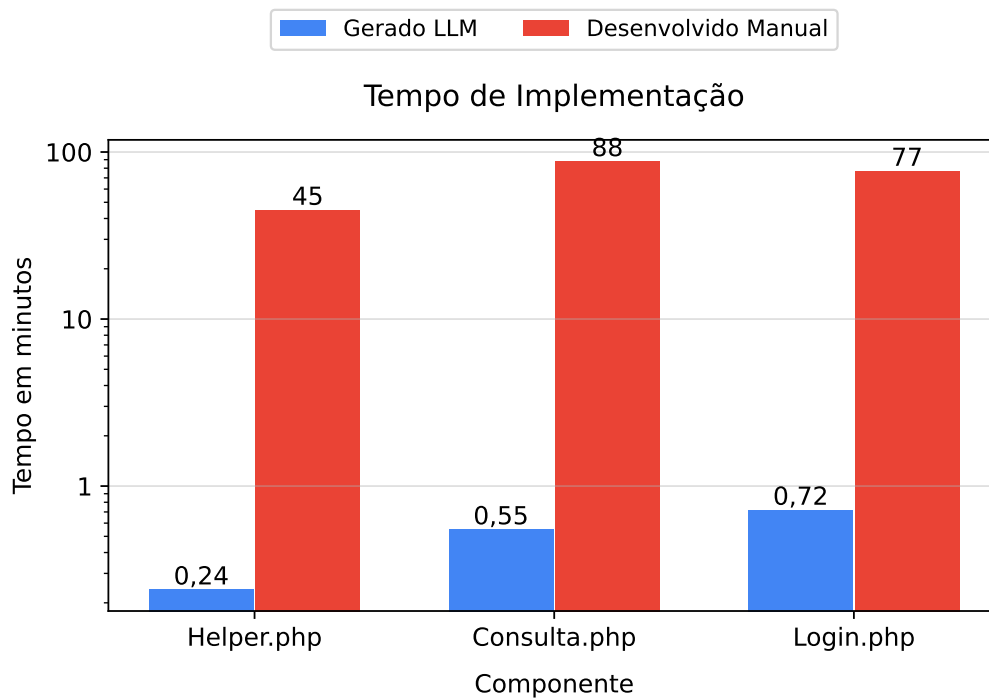
Componente	Tempo LLM	Tempo Manual	Ganho Percentual	Fator de Aceleração
Helper.php	0,24 min	45 min	99,47%	187,50
Consulta.php	0,55 min	88 min	99,38%	160,00
Login.php	0,72 min	77 min	99,06%	106,94

Fonte: próprio autor

A Figura 5.1 apresenta o comparativo dos tempos obtidos entre as abordagens para cada componente. Em função da elevada diferença de magnitude entre os tempos da abordagem manual e da abordagem com LLM, foi adotada uma escala logarítmica para facilitar a visualização dos tempos em cada cenário.

Os resultados apresentados na Tabela 5.1 evidenciam que o uso de LLMs possibilitou ganhos significativos de eficiência. Para o componente *Helper*, o tempo de geração de testes foi reduzido de 45 minutos para 0,24 minutos, representando uma redução de 99,47% no esforço e uma aceleração de aproximadamente 187 vezes em relação ao processo de desenvolvimento manual. Para os componentes *Consulta* e *Login*, observaram-se

Figura 5.1 – Comparação do tempo de implementação e geração de casos de teste



Fonte: próprio autor

também reduções significativas nos tempos de geração dos testes.

5.1.2 Comparação da Cobertura de Código

A Tabela 5.2 apresenta os resultados comparativos entre a geração automatizada de casos de teste com o uso da ferramenta Fatus e a implementação manual dos casos de teste. Foram consideradas as métricas de tempo de desenvolvimento manual, tempo de geração automatizada e as taxas de cobertura de métodos e de linhas de código-fonte.

Tabela 5.2 – Comparação LLM e Desenvolvimento Manual

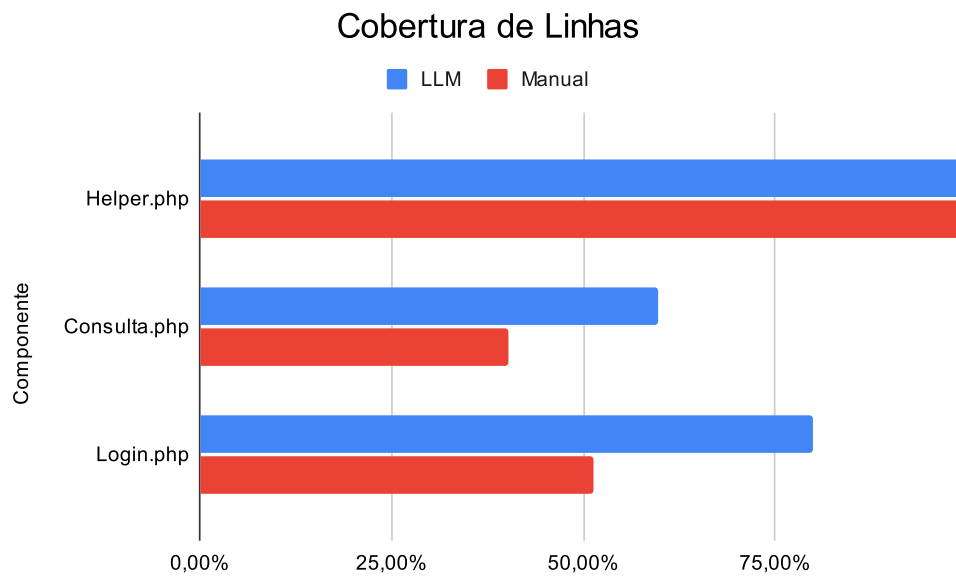
Componente	LLM			Manual		
	Tempo	Métodos	Linhas	Tempo	Métodos	Linhas
Helper.php	0,24 min	100,00%	100,00%	45 min	100,00 %	100,00%
Consulta.php	0,55 min	60,00%	59,65%	88 min	40,00%	40,35%
Login.php	0,72 min	60,00%	80,00%	77 min	45,00%	51,43%

Fonte: próprio autor

A Figura 5.2 apresenta a comparação da cobertura de linhas de código-fonte obtida por cada abordagem para os componentes analisados.

A Figura 5.3 apresenta a comparação da cobertura de métodos obtida por cada

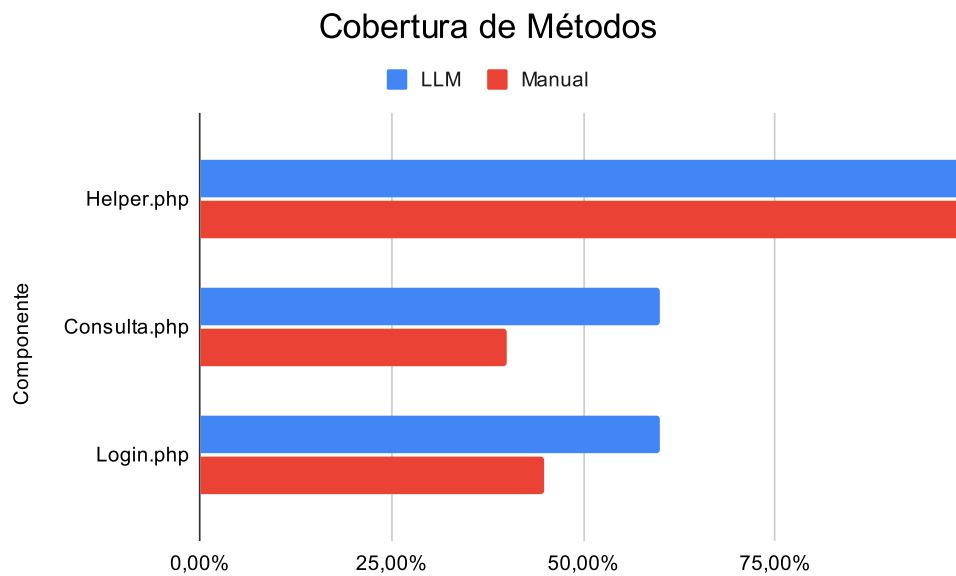
Figura 5.2 – Comparação da cobertura de linhas de código-fonte



Fonte: próprio autor

abordagem para os mesmos componentes analisados.

Figura 5.3 – Comparação da cobertura de métodos



Fonte: próprio autor

5.2 Análise Qualitativa

Nesta seção é apresentada uma análise qualitativa dos casos de teste desenvolvidos manualmente e aqueles gerados automaticamente com o apoio da ferramenta Fatus, que utiliza um modelo de LLM. São apresentadas as observações obtidas durante o experimento de execução dos testes utilizando ambas as abordagens.

5.2.1 Considerações do experimento

No desenvolvimento manual dos casos de teste foi adotado o padrão de implementar apenas um único comando *assert* por método, representando um cenário específico de teste. Essa abordagem permitiu facilitar a identificação dos cenários aprovados ou reprovados e manteve o isolamento dos testes entre si.

Para a execução dos testes manuais foi utilizada a base de dados de desenvolvimento padrão. Não foram implementadas *fixtures* nem outros comandos de preparação que modificassem o banco de dados.

Nos testes gerados automaticamente pela ferramenta Fatus observou-se que, para um mesmo cenário de teste, o modelo frequentemente implementou mais de um comando *assert* no mesmo método. O modelo de LLM não recebeu informações diretas sobre a estrutura do banco de dados e os registros existentes das tabelas. Em alguns casos, os testes gerados implementaram *fixtures* ou registros adicionais no banco de dados, o que provocou conflitos na execução dos testes.

Para contornar essa situação foi realizada a replicação da base de dados para uso exclusivo durante a execução dos testes. Além disso, as tabelas **Consulta** e **Login** precisaram ter seus dados apagados antes da execução dos testes automatizados.

Nas execuções iniciais utilizando a ferramenta, que não foram consideradas no experimento, ocorreram falhas devido à falta de informações sobre a estrutura do banco de dados, configurações do ambiente e utilização de componentes que não estavam configurados. Esses problemas foram mitigados com ajustes no *prompt* e na configuração do ambiente.

De forma geral, os casos de teste gerados pela LLM ampliaram o número de cenários avaliados, quando comparados à abordagem manual, resultando no aumento da cobertura obtida.

Entretanto, a ausência de informações sobre o esquema do banco de dados e o

contexto da aplicação para o modelo de LLM pode ter contribuído para falhas em alguns testes que não puderam ser executados como esperado.

5.2.2 Componente Helper

O componente *Helper* consiste em uma classe PHP que contém métodos auxiliares estáticos utilizados por diferentes classes e componentes da aplicação. Esses métodos são utilizados para formatação de datas, formatação de horários e conversões de strings para ações utilizadas na aplicação.

Os casos de teste desenvolvidos manualmente contemplaram principalmente a:

- a) Verificação dos limites dos parâmetros de data e hora;
- b) Verificação de parâmetros nulos e vazios;
- c) Validação de formatos inválidos de data e hora;
- d) Validação de ordem incorreta de data e hora.

Com a execução da ferramenta Fatus, o modelo de LLM gerou casos de teste que contemplaram principalmente a:

- a) Os mesmos cenários previstos no desenvolvimento manual;
- b) Adição de novos cenários, como verificação de anos bissextos;
- c) Ampliação dos casos de formatos inválidos de data e hora;
- d) Implementação de maior quantidade de asserções por método.

De modo geral, o modelo de LLM apresentou melhor desempenho na geração de variações de cenários de teste adicionais que não haviam sido previstos inicialmente na implementação manual. Esse resultado pode estar relacionado à baixa complexidade do componente *Helper* e ao seu baixo acoplamento com outros componentes do sistema.

5.2.3 Componente Consulta

O componente *Consulta* consiste em uma classe modelo que estende a classe *ActiveRecord* representando o mapeamento objeto-relacional da tabela **Consulta**. Esse mo-

delo é responsável pelo armazenamento das informações referentes às consultas agendadas pelo sistema. Os métodos dessa classe implementam diversas regras de negócio relacionadas ao agendamento das consultas, incluindo verificação de permissões, envio de e-mail e validação de campos recebidos no formulário da aplicação.

Os casos de teste desenvolvidos manualmente contemplaram principalmente a:

- a) Verificação de existência da tabela;
- b) Conversão do formato de data e horários entre o formato brasileiro e o formato do banco de dados (YYYY-mm-dd HH:ii:ss);
- c) Conversão de valores monetários com casas decimais;
- d) Agendamento de horário livre;
- e) Agendamento de horário ocupado;
- f) Permissão de cancelamento para diferentes perfis de usuários;
- g) Permissão de finalização para diferentes perfis de usuários.

Com a execução da ferramenta Fatus, o modelo de LLM gerou casos de teste que contemplaram principalmente a:

- a) Os mesmos cenários previstos no desenvolvimento manual;
- b) Utilização de *fixtures* para criação de registros antes da execução dos testes;
- c) Exclusão dos registros criados após a execução dos testes;
- d) Verificação da definição dos *attributeLabels*;
- e) Inclusão de testes relacionados às regras de validação do formulário (*rules* do Yii);
- f) Testes relacionados ao componente behavior responsável pelo controle de IP;
- g) Ampliação significativa dos cenários de testes de horários disponíveis e ocupados, para o agendamento de consultas;
- h) Verificação de valores limites relacionados aos limites de tolerância (-1, 0, +1);
- i) Verificação de datas anteriores e posteriores ao agendamento;

- j) Validação da existência de registros relacionados nas relações, como usuário responsável pelo agendamento.

De forma geral, o modelo de LLM foi muito assertivo e cobriu cenários adicionais que não haviam sido previstos inicialmente na implementação manual. Em especial houve uma maior abrangência nos cenários de valores limites e validações de formulário. Os testes unitários gerados pelo modelo fizeram o uso de *fixtures* que exigiram ajustes na configuração do ambiente, como limpeza de dados da tabela e definição estática de endereço IP.

5.2.4 Componente Login

O componente *Login* consiste em uma classe modelo que estende a classe *ActiveRecord* e implementa a interface *IdentityInterface*. Esse modelo representa o mapeamento objeto-relacional da tabela **Login** e é responsável pelo gerenciamento das contas de usuários do sistema. Os métodos dessa classe implementam funcionalidades referentes à busca de usuários, validação das credenciais de acesso, redefinição de senha e controle de perfil de usuário no sistema. Além disso, a classe implementa métodos utilizados pelo *framework* Yii para controle de autenticação.

Os casos de teste desenvolvidos manualmente contemplaram principalmente a:

- a) Verificação de existência da tabela;
- b) Verificação de retorno do identificador do usuário;
- c) Validação de senha correta e incorreta;
- d) Verificação da redefinição de senha;
- e) Verificação de perfil de usuário logado para os perfis Veterinário, Funcionário, Cliente e Gerente.

Com a execução da ferramenta Fatus, o modelo de LLM gerou casos de teste que contemplaram principalmente a:

- a) A maioria dos cenários previstos no desenvolvimento manual;
- b) Utilização de *fixtures* para criação de registros antes da execução dos testes;

- c) Exclusão dos registros criados após a execução dos testes;
- d) Verificação da definição dos *attributeLabels*;
- e) Inclusão de testes relacionados às regras de validação do formulário (*rules* do Yii);
- f) Validação de métodos de eventos, como *beforeSave*;
- g) Verificação da existência de usuários com identificador único;
- h) Verificação da criação de novos logins;
- i) Ampliação significativa dos cenários de testes através das combinações de múltiplos comandos *asserts*.

De forma geral, o modelo de LLM foi muito assertivo e cobriu cenários adicionais que não haviam sido previstos inicialmente na implementação manual, como validações de formulário. O único cenário não contemplado na geração dos testes pelo modelo de LLM foi a verificação da senha anterior após a redefinição da senha. Foi observado que alguns testes falharam devido à falta de informações sobre o esquema do banco de dados. Os testes unitários gerados pelo modelo fizeram o uso de *fixtures* que exigiram ajustes na configuração do ambiente, como limpeza de dados da tabela e definição estática de endereço IP.

6 DISCUSSÃO

A ferramenta implementada demonstrou um bom desempenho no tempo de geração de testes unitários, com os testes sendo executados em aproximadamente dois minutos, enquanto o experimento manual demandou cerca de três horas e meia de desenvolvimento. Em relação à cobertura obtida, a ferramenta gerou mais casos de teste e apresentou maior cobertura de linhas e caminhos do que os casos de teste implementados manualmente. A ferramenta também gerou casos de teste não planejados na implementação manual.

Contudo, existem limitações decorrentes do fato de o experimento ter sido realizado utilizando apenas um modelo, o que indica a necessidade de aprofundamento do estudo. Além disso, para o uso em larga escala da ferramenta Fatus, o consumo de *tokens* deverá ser considerado, o que pode gerar custos adicionais e dependência dos provedores dos modelos de LLMs. Alguns casos de testes apresentaram falhas, em especial na necessidade de pós-processamento de marcações geradas pela LLM, bem como na geração de testes unitários que consideram bibliotecas ou configurações de ambiente não presentes no ambiente de execução, como, por exemplo, no uso de *stubs*.

Outra limitação a ser considerada é que modelos de LLMs públicos precisam ter acesso ao código-fonte submetido, o que exige cuidados para evitar que o código seja utilizado para treinamento do modelo ou exposto fora da organização. Isso pode ser mitigado com o uso de modelos locais.

De forma geral, a ferramenta demonstrou redução do esforço necessário para a geração dos testes unitários, em especial na questão do tempo necessário para a geração dos testes em contraste com o tempo de implementação manual, além de a ferramenta automatizada obter maior cobertura de testes. A ferramenta estaria em condições de ser usada por times de desenvolvimento de forma que alguma intervenção humana ainda seja necessária para realizar eventuais ajustes.

7 CONCLUSÕES

Este trabalho permitiu o desenvolvimento de uma ferramenta automatizada que possibilitou a geração de testes unitários com base no código-fonte analisado. Foi apresentado o processo de desenvolvimento da ferramenta, abrangendo engenharia de requisitos, implementação e testes. O trabalho também possibilitou a aplicação de conhecimentos adquiridos nas disciplinas do curso de especialização, tais como Engenharia de Requisitos, Métodos Ágeis de Software, Inteligência Artificial e Ética, Processamento de Linguagem Natural e Metodologia Científica.

A ferramenta implementada foi validada através de um experimento que comparou a implementação manual dos casos de teste com os casos de teste gerados pelo modelo de LLM, no qual foi observado alto desempenho na criação dos casos de teste, além de aumento da cobertura de código (linhas e métodos) do *software* avaliado.

Algumas limitações do experimento incluem o número reduzido de componentes avaliados, o uso de um único modelo de LLM, o que gera dependência do seu desempenho. Além disso, os testes foram implementados pelo próprio autor deste trabalho de conclusão, sendo recomendável aplicar o experimento com um grupo maior de profissionais, com diferentes perfis, para obter resultados mais robustos.

Como trabalhos futuros, o experimento pode ser expandido com a inclusão de novas métricas como cobertura de caminhos, a avaliação de desempenho utilizando mais de um modelo de LLM e a aplicação em equipes de desenvolvimento.

REFERÊNCIAS

- ELDH, S. Transforming Testing: AI as Our Testing Partner . **IEEE Software**, IEEE Computer Society, Los Alamitos, CA, USA, v. 42, n. 04, p. 4–7, jul. 2025. ISSN 1937-4194. Disponível em: <<https://doi.ieeecomputersociety.org/10.1109/MS.2025.3561815>>.
- ELLIMS, M.; BRIDGES, J.; INCE, D. C. The economics of unit testing. **Empirical Software Engineering**, Springer, v. 11, n. 1, p. 5–31, 2006.
- GOMES, A. G. **Sistema de Agendamento de Clínica Veterinária: desenvolvimento de uma aplicação Web utilizando framework Yii 2.0**. Porto Alegre, RS, Brasil: [s.n.], 2023. Trabalho de conclusão de curso (graduação) - Universidade Federal do Rio Grande do Sul, Instituto de Informática. Disponível em: <<http://hdl.handle.net/10183/257998>>.
- JETBRAINS. **Repositório oficial do TestSpark no GitHub**. 2026. Acesso em: 03 abr. 2026. Disponível em: <<https://github.com/JetBrains-Research/TestSpark>>.
- KAUSIK, B. **Efficient LLM Architectures**. 2026. Disponível em: <<https://openreview.net/forum?id=CXdZlCZyTo>>.
- LOPS, A. et al. A system for automated unit test generation using large language models and assessment of generated test suites. In: **2025 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)**. IEEE, 2025. p. 29–36. Disponível em: <<http://dx.doi.org/10.1109/ICSTW64639.2025.10962454>>.
- MARCON, C. J.; FAVARIM, F. O artefato construído pela pesquisa em design science: uma análise à luz da teoria da representação. **Gestão & Produção**, SciELO, v. 28, 2021. ISSN 1806-9649. Disponível em: <<https://www.scielo.br/j/gp/a/3CZmL4JJxLmxCv6b3pnQ8pq/?lang=pt>>.
- RALPH, P. et al. **Empirical Standards for Software Engineering Research: Engineering Research (Design Science)**. 2020. Empirical Standards for Software Engineering Research. Acesso em: 6 mar. 2026. Disponível em: <<https://www2.sigsoft.org/EmpiricalStandards/docs/standards?standard=EngineeringResearch>>.
- RAMOS, A. S. M. Inteligência artificial generativa baseada em grandes modelos de linguagem - ferramentas de uso na pesquisa acadêmica. **SciELO Preprints**, maio 2023. Disponível em: <<https://preprints.scielo.org/index.php/scielo/preprint/view/6105>>.
- RICCA, F. et al. Next-Generation Software Testing: AI-Powered Test Automation . **IEEE Software**, IEEE Computer Society, Los Alamitos, CA, USA, v. 42, n. 04, p. 25–33, jul. 2025. ISSN 1937-4194. Disponível em: <<https://doi.ieeecomputersociety.org/10.1109/MS.2025.3559194>>.
- SCHMIDT, D. C. Software testing in the generative ai era: A practitioner’s playbook. **Computer**, IEEE Computer Society, July 2025.
- TESTPILOT. **Repositório oficial do TestPilot 2 no GitHub**. 2026. Acesso em: 03 abr. 2026. Disponível em: <<https://github.com/neu-se/testpilot2>>.
- VALENTE, M. T. **Engenharia de Software Moderna: Princípios e Práticas para Desenvolvimento de Software com Produtividade**. [s.n.], 2020. ISBN 978-65-00-00077-1. Disponível em: <<https://engsoftmoderna.info/>>.

APÊNDICE A — ARQUIVO DE CONFIGURAÇÃO

Arquivo de Configuração do Fatus

```
{
  "paths": {
    "application" : "C:\\xampp\\htdocs\\sistema\\sacv-web",
    "tests" : "C:\\xampp\\htdocs\\sistema\\sacv-web\\tests",
    "codeception" : "C:\\xampp\\htdocs\\sistema\\sacv-web\\vendor\\bin\\codecept",
    "reportCoverage": "C:\\xampp\\htdocs\\sistema\\sacv-web\\tests\\_output\\coverage"
  },
  "inputs": [
    {
      "file" : "C:\\xampp\\htdocs\\sistema\\sacv-web\\components\\Helper.php",
      "fileCaseTest" : "C:\\xampp\\htdocs\\sistema\\sacv-web\\tests\\unit\\components\\HelperTest.php"
    },
    {
      "file" : "C:\\xampp\\htdocs\\sistema\\sacv-web\\models\\Login.php",
      "fileCaseTest" : "C:\\xampp\\htdocs\\sistema\\sacv-web\\tests\\unit\\models\\LoginTest.php"
    },
    {
      "file" : "C:\\xampp\\htdocs\\sistema\\sacv-web\\models\\Consulta.php",
      "fileCaseTest" : "C:\\xampp\\htdocs\\sistema\\sacv-web\\tests\\unit\\models\\ConsultaTest.php"
    }
  ]
}
```

Figura A.1 – Arquivo de Configuração do Fatus